

Neural Nets

Reference Notebook for Neural Nets. Watch the video lecture for the full breakdown.

The Data

We will use the popular [Boston dataset](#) from the MASS package, which describes some features for houses in Boston in 1978.

- CRIM - per capita crime rate by town
- ZN - proportion of residential land zoned for lots over 25,000 sq.ft.
- INDUS - proportion of non-retail business acres per town.
- CHAS - Charles River dummy variable (1 if tract bounds river; 0 otherwise)
- NOX - nitric oxides concentration (parts per 10 million)
- RM - average number of rooms per dwelling
- AGE - proportion of owner-occupied units built prior to 1940
- DIS - weighted distances to five Boston employment centres
- RAD - index of accessibility to radial highways
- TAX - full-value property-tax rate per 10,000 dollars
- PTRATIO - pupil-teacher ratio by town
- B - $1000(B_k - 0.63)^2$ where B_k is the proportion of blacks by town
- LSTAT - % lower status of the population
- MEDV - Median value of owner-occupied homes in \$1000's

We will be trying to predict the Median Value MEDV

In [4]:

```
library(MASS)
```

In [6]:

```
set.seed(101)
data <- Boston
```

In [8]:

```
str(data)
```

```
'data.frame': 506 obs. of 14 variables:
 $ crim   : num  0.00632 0.02731 0.02729 0.03237 0.06905 ...
 $ zn     : num  18 0 0 0 0 0 12.5 12.5 12.5 12.5 ...
 $ indus  : num  2.31 7.07 7.07 2.18 2.18 2.18 7.87 7.87 7.87 7.87 ...
 $ chas   : int   0 0 0 0 0 0 0 0 0 0 ...
 $ nox    : num  0.538 0.469 0.469 0.458 0.458 0.458 0.524 0.524 0.524 0.524 ...
 $ rm     : num  6.58 6.42 7.18 7 7.15 ...
 $ age    : num  65.2 78.9 61.1 45.8 54.2 58.7 66.6 96.1 100 85.9 ...
 $ dis    : num  4.09 4.97 4.97 6.06 6.06 ...
 $ rad    : int   1 2 2 3 3 3 5 5 5 5 ...
 $ tax    : num  296 242 242 222 222 222 311 311 311 311 ...
 $ ptratio: num  15.3 17.8 17.8 18.7 18.7 18.7 15.2 15.2 15.2 15.2 ...
 $ black  : num  397 397 393 395 397 ...
 $ lstat  : num  4.98 9.14 4.03 2.94 5.33 ...
 $ medv   : num  24 21.6 34.7 33.4 36.2 28.7 22.9 27.1 16.5 18.9 ...
```

In [9]:

```
summary(data)
```

Out[9]:

```
      crim          zn          indus          chas
Min.   : 0.00632    Min.   : 0.00    Min.   : 0.46    Min.   :0.00000
```

```

      crim      crim      crim      crim
1st Qu.: 0.08204 1st Qu.: 0.00 1st Qu.: 5.19 1st Qu.:0.00000
Median : 0.25651 Median : 0.00 Median : 9.69 Median :0.00000
Mean   : 3.61352 Mean   : 11.36 Mean   :11.14 Mean   :0.06917
3rd Qu.: 3.67708 3rd Qu.: 12.50 3rd Qu.:18.10 3rd Qu.:0.00000
Max.   :88.97620 Max.   :100.00 Max.   :27.74 Max.   :1.00000

      nox      rm      age      dis
Min.   :0.3850 Min.   :3.561 Min.   : 2.90 Min.   : 1.130
1st Qu.:0.4490 1st Qu.:5.886 1st Qu.: 45.02 1st Qu.: 2.100
Median :0.5380 Median :6.208 Median : 77.50 Median : 3.207
Mean   :0.5547 Mean   :6.285 Mean   : 68.57 Mean   : 3.795
3rd Qu.:0.6240 3rd Qu.:6.623 3rd Qu.: 94.08 3rd Qu.: 5.188
Max.   :0.8710 Max.   :8.780 Max.   :100.00 Max.   :12.127

      rad      tax      ptratio      black
Min.   : 1.000 Min.   :187.0 Min.   :12.60 Min.   : 0.32
1st Qu.: 4.000 1st Qu.:279.0 1st Qu.:17.40 1st Qu.:375.38
Median : 5.000 Median :330.0 Median :19.05 Median :391.44
Mean   : 9.549 Mean   :408.2 Mean   :18.46 Mean   :356.67
3rd Qu.:24.000 3rd Qu.:666.0 3rd Qu.:20.20 3rd Qu.:396.23
Max.   :24.000 Max.   :711.0 Max.   :22.00 Max.   :396.90

      lstat      medv
Min.   : 1.73 Min.   : 5.00
1st Qu.: 6.95 1st Qu.:17.02
Median :11.36 Median :21.20
Mean   :12.65 Mean   :22.53
3rd Qu.:16.95 3rd Qu.:25.00
Max.   :37.97 Max.   :50.00

```

In [10]:

```
head(data)
```

Out[10]:

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	black	lstat	medv
1	0.00632	18	2.31	0	0.538	6.575	65.2	4.09	1	296	15.3	396.9	4.98	24
2	0.02731	0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.9	9.14	21.6
3	0.02729	0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
4	0.03237	0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
5	0.06905	0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.9	5.33	36.2
6	0.02985	0	2.18	0	0.458	6.43	58.7	6.0622	3	222	18.7	394.12	5.21	28.7

In [12]:

```
any(is.na(data))
```

Out[12]:

FALSE

Neural Net Model

First you'll need to install the neural net library:

In [2]:

```
#install.packages('neuralnet',repos = 'http://cran.us.r-project.org')
```

In [15]:

```
library(neuralnet)
```

Training the Model

As a first step, we are going to address data preprocessing. It is good practice to normalize your data before training a neural network. Depending on your dataset, avoiding normalization may lead to useless results or to a very difficult training process (most of the times the algorithm will not converge before the number of maximum iterations allowed). You can choose different methods to scale the data (z-normalization, min-max scale, etc...). Usually scaling in the intervals [0,1] or [-1,1] tends to give better results. We therefore scale and split the data before moving on:

In [19]:

```
maxs <- apply(data, 2, max)
mins <- apply(data, 2, min)
```

In [20]:

```
maxs
```

Out[20]:

```
crim
88.9762
zn
100
indus
27.74
chas
1
nox
0.871
rm
8.78
age
100
dis
12.1265
rad
24
tax
711
ptratio
22
black
396.9
lstat
37.97
medv
50
```

In [21]:

```
mins
```

Out[21]:

```
crim
0.00632
zn
0
indus
0.46
chas
0
```

nox
0.385
rm
3.561
age
2.9
dis
1.1296
rad
1
tax
187
ptratio
12.6
black
0.32
lstat
1.73
medv
5

In [17]:

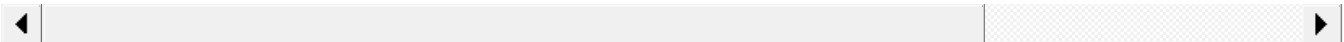
```
scaled <- as.data.frame(scale(data, center = mins, scale = maxs - mins))
```

In [22]:

```
head(scaled)
```

Out[22]:

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio
1	0	0.18	0.06781525	0	0.3148148	0.5775053	0.6416066	0.2692031	0	0.2080153	0.287234
2	0.0002359225	0	0.2423021	0	0.1728395	0.5479977	0.7826982	0.348962	0.04347826	0.1049618	0.5531915
3	0.0002356977	0	0.2423021	0	0.1728395	0.6943859	0.5993821	0.348962	0.04347826	0.1049618	0.5531915
4	0.0002927957	0	0.06304985	0	0.1502058	0.6585553	0.4418126	0.4485446	0.08695652	0.06679389	0.6489362
5	0.0007050701	0	0.06304985	0	0.1502058	0.6871048	0.5283213	0.4485446	0.08695652	0.06679389	0.6489362
6	0.0002644715	0	0.06304985	0	0.1502058	0.5497222	0.5746653	0.4485446	0.08695652	0.06679389	0.6489362



Train and Test Sets

Now with our standardized data, let's split it:

In [23]:

```
library(caTools)
split = sample.split(scaled$medv, SplitRatio = 0.70)

train = subset(scaled, split == TRUE)
test = subset(scaled, split == FALSE)
```

Training the Model

In [31]:

```
# Call package
library(neuralnet)
```

Formula for Neural Net

For some odd reasons, the `neuralnet()` function won't accept a formula in the form: `y~.` that we are used to using. Instead you have to call all the columns added together. Here is some convenience code to help quickly create that formula:

In [34]:

```
# Get column names
n <- names(train)
```

In [35]:

```
n
```

Out[35]:

```
'crim' 'zn' 'indus' 'chas' 'nox' 'rm' 'age' 'dis' 'rad' 'tax' 'ptratio' 'black' 'lstat' 'medv'
```

In [36]:

```
# Paste together
f <- as.formula(paste("medv ~", paste(n[!n %in% "medv"], collapse = " + ")))
```

In [37]:

```
f
```

Out[37]:

```
medv ~ crim + zn + indus + chas + nox + rm + age + dis + rad +
      tax + ptratio + black + lstat
```

In [38]:

```
nn <- neuralnet(f,data=train,hidden=c(5,3),linear.output=TRUE)
```

Neural Net Visualization

You can plot out your model to see a very neat visualization with the weights on each connection.

The black lines show the connections between each layer and the weights on each connection while the blue lines show the bias term added in each step. The bias can be thought as the intercept of a linear model. The net is essentially a black box so we cannot say that much about the fitting, the weights and the model. Suffice to say that the training algorithm has converged and therefore the model is ready to be used.

In [1]:

```
#plot(nn)
```

Predictions using the Model

Now we can try to predict the values for the test set and calculate the MSE. Remember that the net will output a normalized prediction, so we need to scale it back in order to make a meaningful comparison (or just a simple prediction).

In [43]:

```
# Compute Predictions off Test Set
```

```
predicted.nn.values <- compute(nn,test[1:13])
```

In [45]:

```
# Its a list returned
str(predicted.nn.values)
```

```
List of 2
 $ neurons      :List of 3
  ..$ : num [1:139, 1:14] 1 1 1 1 1 1 1 1 1 1 ...
  .. ..- attr(*, "dimnames")=List of 2
  .. .. ..$ : chr [1:139] "4" "6" "8" "9" ...
  .. .. ..$ : chr [1:14] "1" "crim" "zn" "indus" ...
  ..$ : num [1:139, 1:6] 1 1 1 1 1 1 1 1 1 1 ...
  .. ..- attr(*, "dimnames")=List of 2
  .. .. ..$ : chr [1:139] "4" "6" "8" "9" ...
  .. .. ..$ : NULL
  ..$ : num [1:139, 1:4] 1 1 1 1 1 1 1 1 1 1 ...
  .. ..- attr(*, "dimnames")=List of 2
  .. .. ..$ : chr [1:139] "4" "6" "8" "9" ...
  .. .. ..$ : NULL
 $ net.result: num [1:139, 1] 0.552 0.422 0.342 0.252 0.365 ...
  ..- attr(*, "dimnames")=List of 2
  .. ..$ : chr [1:139] "4" "6" "8" "9" ...
  .. ..$ : NULL
```

In [46]:

```
# Convert back to non-scaled predictions
true.predictions <- predicted.nn.values$net.result*(max(data$medv)-min(data$medv))+min(data$medv)
```

In [47]:

```
# Convert the test data
test.r <- (test$medv)*(max(data$medv)-min(data$medv))+min(data$medv)
```

In [48]:

```
# Check the Mean Squared Error
MSE.nn <- sum((test.r - true.predictions)^2)/nrow(test)
```

In [49]:

```
MSE.nn
```

```
[1] 20.45308118
```

Visualize Error

In [51]:

```
error.df <- data.frame(test.r,true.predictions)
```

In [52]:

```
head(error.df)
```

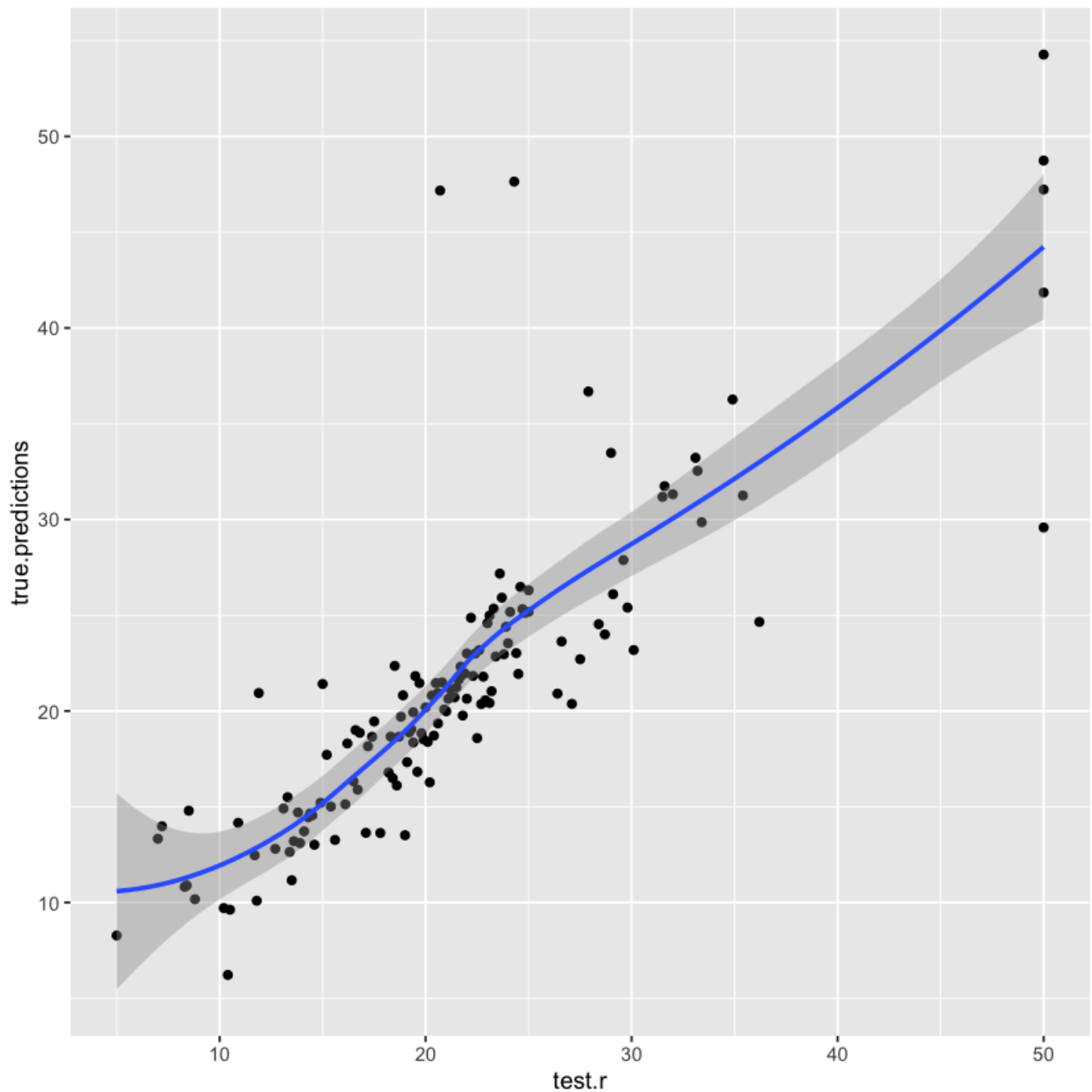
Out[52]:

	test.r	true.predictions
1	22.4	20.8568624

4	33.4	29.8308021
6	28.7	23.99832638
8	27.1	20.37098111
9	16.5	16.32652695
11	15	21.412632
14	20.4	18.71094098

In [55]:

```
library(ggplot2)
ggplot(error.df, aes(x=test.r, y=true.predictions)) + geom_point() + stat_smooth()
```



Looks like a few houses threw off our model, but overall its not looking too bad considering we're pretty much treating it like a total black box.

Conclusion

Neural networks resemble black boxes a lot: explaining their outcome is much more difficult than explaining the outcome of simpler

model such as a linear model. Therefore, depending on the kind of application you need, you might want to take into account this factor too. Furthermore, as you have seen above, extra care is needed to fit a neural network and small changes can lead to different results.